



TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
NATIONAL COLLEGE OF ENGINEERING

A Minor Project Report

On

Customer Segmentation and Intelligent Mobile Recommendation System

Submitted By:

Aayush Chhuka (NCE080BCT002)

Hom Raj Bhandari (NCE080BCT016)

Sudip Kumar Tamang (NCE080BCT042)

Suresh Khadka (NCE080BCT046)

Submitted To:

Department of Electronics & Computer Engineering

August, 2026

Page of Approval

TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
NATIONAL COLLEGE OF ENGINEERING
DEPARTMENT OF ELECTRONICS AND COMPUTER ENGINEERING

The undersigned certifies that they have read and recommended to the Institute of Engineering for acceptance of a project report entitled ”**Customer Segmentation and Intelligent Mobile Recommendation System**” submitted by **Aayush Chhuka, Hom Raj Bhandari, Sudip Kumar Tamang, Suresh Khadka** in partial fulfillment of the requirements for the Bachelor’s degree in Electronics, Information and Communication Engineering & Computer Engineering.

.....

Supervisor

Er. Subash Pandey

Deputy Head of Department

Department of Electronics and Computer

Engineering,

National College of Engineering, IOE, TU.

.....

External examiner

Full Name

Designation

Affiliation

Date of approval:

Copyright

The author has agreed that the Library, Department of Electronics and Computer Engineering, National College of Engineering, and Institute of Engineering may make this report freely available for inspection. Moreover, the author has agreed that permission for extensive copying of this project report for scholarly purposes may be granted by the supervisors who supervised the work recorded herein or, in their absence, by the Head of the Department wherein the project report was done. It is understood that recognition will be given to the author of this report and the Department of Electronics and Computer Engineering, National College of Engineering, IOE for any use of the material of this project report. Copying publication or the other use of this report for financial gain without the approval of to the Department of Electronics and Computer Engineering, National College of Engineering, Institute of Engineering, and the author's written permission is prohibited.

Request for permission to copy or to make any other use of the material in this report in whole or in part should be addressed to:

Head
Department of Electronics and Computer Engineering
National College of Engineering
Talchhikhel, Lalitpur.

Acknowledgements

We would like to express our sincere gratitude to the Department of Electronics and Computer Engineering at National College of Engineering for providing us with the opportunity, facilities, and support needed to complete this minor project. The guidance and learning environment provided by the department helped us throughout the development of this project.

We would like to sincerely thank our project supervisor, **Er. Subash Pandey**, for his continuous guidance, valuable suggestions, and patient support during every stage of the project. His feedback helped us improve our work and solve many technical and practical problems. His encouragement motivated us to complete the project with confidence.

Finally, we express our heartfelt thanks to our families, friends, and classmates for their constant encouragement, patience, and moral support throughout this journey. We are thankful to everyone who directly or indirectly contributed to the successful completion of this minor project.

Abstract

The rapid growth of the smartphone market has provided customers with a wide range of devices that differ in price, specifications, performance, and features. This makes it difficult for customers to identify a suitable smartphone and for retailers to understand and respond to different customer preferences and purchasing behaviours. This project presents a hybrid mobile recommendation system that combines customer segmentation, content based recommendation, and collaborative filtering to provide personalised and explainable smartphone recommendations.

The system uses smartphone specification data from GSMArena and performance data from AnTuTu together with customer profile and interaction data. An XGBoost regression model is used to predict smartphone performance from engineered hardware features. The model achieves an R^2 score of approximately 0.88 on the held out test set, with a mean absolute error of approximately 0.18 on the log transformed target. SHAP is used to explain model predictions and identify important features influencing smartphone performance. For customer segmentation, K-Means clustering is applied to engineered customer features. Four customer segments are identified: Hardcore Gamer, Mainstream Mid Range Shopper, Premium Flagship Shopper, and Budget Buyer. The segmentation achieved a silhouette score of 0.1718.

To improve personalised recommendations, a separate collaborative filtering pipeline is developed using 4,557 customers, 1,787 smartphones, and 86,195 interaction records covering views, searches, comparisons, wishlists, purchases, and ratings. Several collaborative filtering approaches are evaluated, with the SVD and item cosine hybrid achieving the best NDCG@10 of 0.0507.

The complete system is implemented as a full stack web application using React, Node.js, Express, Prisma, PostgreSQL, and FastAPI, with Docker Compose for service orchestration. The proposed system provides a foundation for personalised, explainable, and behaviour aware smartphone recommendations, with scope for further improvement through real user interactions and continuous model updating.

Keywords: *Mobile Recommendation System, Customer Segmentation, Collaborative Filtering, XGBoost, K-Means Clustering*

Contents

Page of Approval	ii
Copyright	iii
Acknowledgements	iv
Abstract	v
Contents	viii
List of Figures	ix
List of Tables	x
List of Abbreviations	xi
1 Introduction	1
1.1 Background	1
1.2 Problem Statement	3
1.3 Aim and Objectives	3
1.4 Scope	4
2 Literature Review	5
2.1 Related Work	5
2.2 Related Theory	8
2.2.1 Gradient Boosting	8
2.2.2 XGBoost	8
2.2.3 SHAP (SHapley Additive exPlanations)	9
2.2.4 K-Means Clustering	9
2.2.5 Customer Segmentation	9
2.2.6 Collaborative Filtering	9
2.2.7 Content Based Recommendation	10
2.2.8 Hybrid Recommendation System	10

2.2.9	Singular Value Decomposition (SVD)	10
2.2.10	Cosine Similarity	11
3	Methodology	12
3.1	Block Diagram of the System	12
3.1.1	Data Collection	12
3.1.2	Data cleaning and Preprocessing	15
3.1.3	Feature Engineering	17
3.1.4	Scaling, Outlier Handling, and Train Test Split	18
3.1.5	Segmentation Model	19
3.1.6	Recommendation Engine	20
3.2	Methods and Algorithms	21
3.2.1	XGBoost Algorithm	21
3.2.2	K-Means Clustering for Customer Segmentation	23
3.3	Evaluation Metrics	24
3.3.1	Metrics for the Regressor	24
3.3.2	Metrics for the Segmentation	27
3.3.3	Metrics for the Recommendation Engine	27
4	Experimental Setup	29
4.1	Hardware Configuration	29
4.2	Software and Tools	29
5	System Design	31
5.1	Requirement Specification	31
5.1.1	Functional Requirements	31
5.1.2	Non-Functional Requirements	31
5.2	System Architecture	32
5.3	Use Case Diagram	34
5.4	Data Flow Diagram	35
5.5	Iterative Development Model	37
6	Results & Discussion	38
6.1	Experiment Setup	38
6.2	Data Preparation Results	38
6.3	Machine Learning Results	39
6.4	Recommendation System Output	43

6.5	Software Development Approach	43
6.6	Model Deployment	44
6.7	User Interface and Backend	44
6.8	Testing, Evaluation, and Validation	44
7	Conclusions	46
8	Limitations and Future Enhancement	47
8.1	Limitations	47
8.2	Future Enhancement	47
	References	47
	Appendices	51

List of Figures

3.1	Block diagram of the System	12
3.2	Data Collection and Integration Process	13
3.3	Data Processing Flow	15
3.4	Customer Segmentation Flow	19
3.5	Recommendation Engine Workflow	20
3.6	XGBoost Ensemble of Decision Trees [1]	22
3.7	K-Means Clustering of Customers [2]	24
5.1	System Architecture of the Mobile Recommendation System	32
5.2	Use Case Diagram of the Mobile Recommendation System	34
5.3	Level 0 Data Flow Diagram	35
5.4	Level 1 Data Flow Diagram	36
5.5	Iterative Development Model	37
6.1	Visualizations of XGBoost Regressor Performance on the Test Set	42
6.2	SHAP Feature Importance Summary Plot	43
6.3	Frontend User Interface of the Mobile Recommendation System	44

List of Tables

6.1	Phone Dataset Before Processing	38
6.2	Phone Dataset After Processing	39
6.3	XGBoost Model Hyperparameters	40
6.4	XGBoost Training and Testing Performance Metrics	40
6.5	5-Fold Cross-Validation Results	41

List of Abbreviations

API	Application Programming Interface
BCT	Bachelor in Computer Engineering
CPU	Central Processing Unit
CSS	Cascading Style Sheet
CSV	Comma Separated Values
DBSCAN	Density-Based Spatial Clustering of Applications with Noise
GPU	Graphics Processing Unit
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
ID	Identifier
JSON	JavaScript Object Notation
LIME	Local Interpretable Model-agnostic Explanations
MAE	Mean Absolute Error
MAPE	Mean Absolute Percentage Error
ML	Machine Learning
MSE	Mean Squared Error
NCE	National College of Engineering
ORM	Object Relational Mapper
OTP	One Time Password
PCA	Principal Component Analysis
RAM	Random Access Memory
REST	Representational State Transfer
RMSE	Root Mean Squared Error
RMSLE	Root Mean Squared Logarithmic Error
SHAP	SHapley Additive exPlanations
SQL	Structured Query Language
UI	User Interface
XAI	Explainable Artificial Intelligence
XGBoost	Extreme Gradient Boosting

1. Introduction

1.1 Background

The mobile phone market is one of the most competitive consumer electronics industries, with new smartphones being introduced regularly by brands such as Samsung, Apple, Xiaomi, Vivo, Oppo, and Realme. These devices differ in terms of price, hardware specifications, performance, camera capabilities, battery capacity, display, software, and other features. The wide range of available choices makes it difficult for customers to identify a smartphone that matches their needs and budget. At the same time, mobile retailers need to understand customer preferences and purchasing behaviour to provide suitable recommendations and improve customer engagement.

Customer segmentation is a widely used approach for understanding differences among customers by grouping individuals with similar characteristics, preferences, and behaviour [3]. Instead of treating all customers in the same way, segmentation allows retailers to identify groups with common patterns and provide more relevant recommendations and services. In the smartphone market, customers may differ considerably in their priorities. Some may focus on gaming performance and display quality, while others may prioritise camera capabilities, battery life, storage, price, or brand preference. Identifying these groups can therefore support personalised recommendations and better customer analysis.

The increasing availability of customer and product data has made intelligent recommendation systems more practical. Customer activity can provide information about searches, browsing behaviour, comparisons, wishlists, purchases, and ratings, while smartphone datasets provide detailed information about device specifications. GSMArena provides extensive smartphone specification data [4], while AnTuTu benchmark scores provide an indication of smartphone performance [5]. Combining customer behaviour with smartphone characteristics allows a recommendation system to consider both what customers prefer and what smartphones offer.

However, recommendation systems face several challenges. Content based approaches can recommend products with similar characteristics but may provide limited personalisation when sufficient user behaviour is unavailable. Collaborative filtering can learn from interactions between customers and smartphones, but it requires sufficient historical interaction

data and faces the cold start problem for new customers with little or no interaction history [6]. Customer segmentation can provide additional information about groups of similar customers, but clustering alone does not directly determine which individual smartphones should be recommended. Therefore, combining these approaches can provide a more comprehensive recommendation strategy.

This project addresses these challenges by developing a hybrid mobile recommendation system that combines smartphone characteristics, customer segmentation, and behavioural information. The project uses a cleaned GSMArena smartphone catalog together with AnTuTu performance data and customer profile and interaction data. A dedicated data preparation pipeline generates an interaction log containing 86,195 interaction records from 4,557 customers and 1,787 smartphones. The interaction data includes views, searches, comparisons, wishlists, purchases, and ratings, providing a richer representation of customer behaviour for collaborative filtering.

The machine learning component uses the XGBoost algorithm, a gradient boosting technique suitable for structured data [7], to predict smartphone performance from engineered hardware features. The trained model achieves an R^2 score of approximately 0.88 on the held out test set, with a mean absolute error of approximately 0.18 on the log transformed target. SHAP analysis is used to explain individual model predictions and identify the features that have the greatest influence on predicted smartphone performance.

For customer segmentation, the project applies the K-Means clustering algorithm to engineered customer features. The clustering process uses numerical and categorical customer characteristics to identify groups with similar preferences and purchasing patterns. The optimal number of clusters is selected using the silhouette score, with four clusters producing the highest silhouette score of 0.1718. The resulting customer groups are labelled as Hardcore Gamer, Mainstream Mid Range Shopper, Premium Flagship Shopper, and Budget Buyer. These segments provide a customer persona that can be incorporated into the recommendation process, particularly when limited interaction history is available.

The project also implements a collaborative filtering pipeline to learn relationships between customers and smartphones from historical interactions. Several approaches, including item item cosine similarity, Singular Value Decomposition (SVD), and Implicit Alternating Least Squares (ALS), are evaluated. A hybrid model combining SVD and item cosine similarity achieves the best NDCG@10 score of 0.0507. To address cold start situations, the system uses a fallback strategy based on customer cluster popularity, province popularity, district popularity, and global popularity, allowing recommendations to be generated even when

sufficient individual interaction history is unavailable.

The complete system is implemented as a web application using React and Vite for the frontend, Node.js and Express for the backend, Prisma ORM and PostgreSQL for data management, and FastAPI for machine learning services. Authentication and role based access control are included to provide separate functionalities for customers, and administrators. Docker Compose is used to manage and orchestrate the different services of the application.

Overall, the project integrates data collection, preprocessing, feature engineering, customer segmentation, predictive modelling, collaborative filtering, and web application development into a single hybrid recommendation system. By combining customer segments with content based and behaviour based recommendation approaches, the system aims to provide more personalised and explainable smartphone recommendations while supporting data driven decision making for mobile retailers.

1.2 Problem Statement

The mobile phone market offers a wide range of smartphones with different prices, features, and specifications. This makes it difficult for retailers to understand customer preferences and recommend the most suitable mobile phones. Customer preferences are influenced by factors such as performance requirements, camera quality, battery life, budget, and brand preference, which cannot be accurately identified using basic demographic information alone.

Retailers also face challenges in grouping customers based on their purchasing behaviour and using this information to support product recommendations and marketing decisions. Without an intelligent customer segmentation system, recommendations and promotional activities may not effectively match the interests of different customer groups.

Therefore, there is a need to develop a machine learning based customer segmentation system that can analyse customer and product data, identify meaningful customer groups, and support intelligent mobile phone recommendations.

1.3 Aim and Objectives

The aim of this project is to develop an intelligent mobile recommendation system using machine learning based customer segmentation that recommends suitable smartphones based on customer preferences.

The objectives of the project are as follows:

- I. To collect and preprocess mobile phone data from GSMArena and AnTuTu benchmark through web scraping for recommendation and analysis.

- II. To develop a machine learning based hybrid recommendation system using customer preferences, segmentation, and interaction behaviour.
- III. To evaluate the performance measure and accuracy of the proposed system.
- IV. To develop a secure web application with user authentication and database management.

1.4 Scope

This project focuses on developing a hybrid mobile phone recommendation system that uses smartphone specifications, customer preferences, and customer interaction behaviour to provide personalised recommendations.

The project includes customer segmentation using K-Means clustering to identify groups of customers with similar preferences and purchasing behaviour. It also includes collaborative filtering using customer–phone interaction data, which is combined with content based recommendation to improve personalisation. A cold start strategy is included to provide recommendations for customers with limited or no interaction history.

The system is implemented as a web based application with user authentication, role based access control, database management, smartphone search and filtering, comparison, wishlist, and recommendation features. The application uses React for the frontend, Node.js and Express for the backend, PostgreSQL for data management, and FastAPI for machine learning services.

The scope of the project is limited to smartphone recommendation and customer analysis. It does not include real time inventory, live pricing, or a dedicated mobile application. The system can be further improved by incorporating real customer interaction data and continuously updating the recommendation models.

2. Literature Review

The review of existing systems and methods to position the current work relative to previous research are as follow:

2.1 Related Work

Chen and Guestrin introduced XGBoost, a scalable gradient boosting framework designed for efficient learning on structured datasets [7]. The model constructs an ensemble of decision trees in a sequential manner, where each tree corrects the errors of the previous ones while incorporating regularisation techniques to reduce overfitting. XGBoost also supports parallel computation, handles missing values automatically, and efficiently processes high-dimensional data. Experimental evaluations on multiple benchmark datasets demonstrated that XGBoost consistently achieved higher prediction accuracy and significantly faster training speed than existing gradient boosting implementations. Owing to these advantages, XGBoost has become one of the most widely adopted algorithms in recommendation systems, customer analytics, fraud detection, and predictive modelling. In this project, XGBoost is employed to predict the performance score of mobile phones from their hardware specifications.

Recent studies have combined XGBoost with SHAP (SHapley Additive exPlanations) to build explainable recommendation systems [8]. These studies utilise customer transaction history, demographic information, browsing behaviour, and purchase records to predict customer purchasing decisions using XGBoost. SHAP is then applied to quantify the contribution of every input feature toward the final prediction, allowing both businesses and customers to understand why a recommendation was generated. Experimental results reported that the integration of SHAP significantly improves model transparency without sacrificing predictive performance. The studies concluded that explainable recommendation systems increase user trust, simplify model interpretation, and support better business decision-making. This project adopts a similar explainable AI approach by applying SHAP to interpret the influence of different mobile phone specifications on the predicted performance score.

İşık proposed a hyper-personalised recommendation framework for fashion retail by integrating customer segmentation with recommendation algorithms [9]. The proposed system first performs customer segmentation using RFM (Recency, Frequency, Monetary) analy-

sis together with K-Means clustering. Customer location information is then incorporated into the segmentation process, while association rules generated using the Apriori algorithm are employed to produce personalised product recommendations. Experimental evaluation showed that incorporating geographical information alongside purchasing behaviour improved recommendation relevance and personalisation compared to conventional segmentation methods. The study demonstrated that combining multiple customer characteristics enables businesses to better understand customer preferences and provide more targeted recommendations. These findings inspired the customer segmentation stage of the proposed recommendation system.

Joung and Kim presented an interpretable machine learning framework for customer segmentation that emphasises feature importance rather than relying solely on demographic characteristics [10]. Their approach extracts product-related features from customer reviews and purchasing behaviour and performs customer segmentation using machine learning techniques. Experimental results indicated that explainable segmentation generates more meaningful customer groups and provides valuable business insights regarding customer preferences and purchasing patterns. The authors concluded that integrating explainable AI with customer segmentation improves model interpretability and supports strategic marketing decisions. Their work provides the motivation for incorporating explainability into the customer analysis component of this project.

A recent study proposed a SHAP-enhanced customer segmentation framework for retail analytics by combining advanced feature engineering, Principal Component Analysis (PCA), and multiple clustering algorithms [11]. The framework compares clustering methods including K-Means, DBSCAN, Agglomerative Clustering, and Gaussian Mixture Models before selecting the best-performing algorithm using clustering evaluation metrics. Experimental results demonstrated that combining dimensionality reduction with explainable clustering improves cluster quality, enhances interpretability, and supports more informed business decisions. The study highlighted that explainable clustering enables organisations to better understand customer behaviour while maintaining high analytical performance.

Value-aware recommendation systems have emerged as an important direction in recommender system research [12]. Unlike traditional recommendation systems that focus only on customer preferences, these approaches simultaneously consider customer lifetime value, purchasing frequency, and revenue contribution during recommendation generation. Customers are segmented according to their business value, after which different recommendation strategies are applied to maximise both customer satisfaction and organisational profitability. Experimental studies reported improvements in customer retention, recommendation

effectiveness, and overall business revenue when customer value was incorporated into recommendation strategies. The findings emphasise that recommendation systems should optimise not only recommendation accuracy but also long-term business objectives.

Zhang *et al.* proposed a large-scale purchase prediction framework using customer behavioural data collected from the Tmall Recommendation Prize competition [13]. The framework utilises click history, shopping cart activities, purchase records, and user interaction features to predict future purchasing behaviour through ensemble machine learning models. Extensive experiments demonstrated that behavioural features substantially improve purchase prediction accuracy compared with demographic information alone. The authors concluded that customer interaction data provides valuable information for understanding purchasing intentions and significantly enhances recommendation performance in large-scale e-commerce environments. Their work highlights the importance of incorporating behavioural features into recommendation systems.

Several studies in predictive analytics for e-commerce have investigated the effectiveness of customer browsing behaviour, clickstream data, and historical transaction records for predicting future purchasing activities [14]. These studies compared traditional machine learning algorithms such as Logistic Regression with ensemble methods including Random Forest, Gradient Boosting, and XGBoost. Experimental results consistently showed that tree-based ensemble models outperform linear models in terms of prediction accuracy, precision, recall, and robustness when handling complex customer behaviour. The research further demonstrated that combining multiple behavioural features enables businesses to identify purchasing intentions more accurately, thereby improving recommendation quality and customer engagement.

Customer segmentation combined with personalised recommendation has become one of the most widely adopted approaches in modern recommender systems [15]. In these systems, customers are first grouped into homogeneous clusters using techniques such as K-Means clustering based on purchasing behaviour, demographic characteristics, or RFM features. Separate recommendation strategies are then developed for each customer segment. Experimental evaluations consistently reported higher recommendation accuracy, improved personalisation, increased customer satisfaction, and better marketing effectiveness compared with traditional recommendation approaches. These findings demonstrate the effectiveness of integrating customer segmentation into recommendation systems.

Recent explainable recommendation frameworks combine recommendation algorithms with explainable AI techniques such as SHAP, LIME, and knowledge graph-based explanations

to improve transparency and user trust [16]. These systems not only generate personalised product recommendations but also provide understandable explanations by identifying the most influential customer and product features responsible for each recommendation. Experimental results indicate that explainable recommendations improve user confidence, increase acceptance of recommendations, and assist businesses in validating model decisions. The studies conclude that explainability is becoming an essential component of modern recommendation systems, particularly in applications where transparency and trust are critical.

The reviewed literature demonstrates that XGBoost provides highly accurate and computationally efficient prediction for structured datasets, making it well suited for recommendation and predictive analytics applications. The integration of SHAP further enhances model transparency by explaining feature contributions without reducing predictive performance. Existing research also shows that customer segmentation using K-Means and RFM analysis significantly improves recommendation relevance by grouping customers with similar purchasing behaviours and preferences. Furthermore, behavioural features such as browsing history, clickstream data, and transaction records consistently improve purchase prediction accuracy, while explainable recommendation techniques increase user trust and business interpretability. These findings collectively provide a strong foundation for the proposed mobile phone recommendation system, which integrates XGBoost-based performance prediction, K-Means customer segmentation, and SHAP-based explainability to deliver accurate, personalised, and interpretable recommendations.

2.2 Related Theory

2.2.1 Gradient Boosting

Gradient Boosting is an ensemble machine learning technique that combines multiple weak decision trees to create a strong predictive model [17]. The trees are built sequentially, where each new tree learns from the errors made by the previous trees. This process improves prediction accuracy and allows the model to capture complex patterns in structured data.

2.2.2 XGBoost

Extreme Gradient Boosting (XGBoost) is an optimised implementation of the Gradient Boosting algorithm [7]. It is widely used because of its high accuracy, fast training speed, and ability to handle structured datasets. XGBoost also includes regularisation techniques that help reduce overfitting and can handle missing values efficiently.

In this project, XGBoost is used to predict smartphone performance scores from engineered smartphone specifications. The predicted performance information is then used as part of

the smartphone scoring and recommendation process.

2.2.3 SHAP (SHapley Additive exPlanations)

SHAP (SHapley Additive exPlanations) is an Explainable Artificial Intelligence technique used to interpret machine learning predictions [18]. It assigns an importance value to each feature, showing how the feature contributes to increasing or decreasing a prediction.

In this project, SHAP is used to explain the predictions made by the XGBoost model and identify the smartphone features that have the greatest influence on predicted performance.

2.2.4 K-Means Clustering

K-Means Clustering is an unsupervised machine learning algorithm used to group similar data points into a fixed number of clusters (K) [19]. It assigns each data point to the nearest cluster centroid and then updates the centroids based on the mean of the assigned points. This process continues until the clusters become stable.

In this project, K-Means is applied to engineered customer features to identify groups of customers with similar preferences and behaviour. Four customer segments are identified: Hardcore Gamer, Mainstream Mid Range Shopper, Premium Flagship Shopper, and Budget Buyer.

2.2.5 Customer Segmentation

Customer segmentation is the process of dividing customers into groups based on shared characteristics, preferences, or behaviour. It allows a recommendation system to treat different groups of customers according to their specific interests rather than using the same approach for every customer [10].

In this project, customer segmentation is performed using K-Means clustering. The resulting customer segments are used to understand customer behaviour and support personalised recommendations, particularly when limited individual interaction data is available.

2.2.6 Collaborative Filtering

Collaborative Filtering is a recommendation technique that learns customer preferences from interactions between users and items. Instead of relying only on product specifications, it uses behavioural patterns such as views, searches, comparisons, wishlists, purchases, and ratings to identify items that may be relevant to a customer [20].

Collaborative filtering can identify relationships between users and items based on their previous interactions. This makes it useful for recommendation systems where sufficient

user interaction data is available.

In this project, collaborative filtering is trained using customer–phone interaction data. Different collaborative filtering approaches are evaluated to identify the model that provides the most effective ranking of smartphones.

2.2.7 Content Based Recommendation

Content Based Recommendation recommends items by comparing their characteristics with the preferences or interests of a user [21]. It focuses on the similarity between user preferences and item features rather than relying only on interactions from other users.

In this project, smartphone features such as brand, RAM, storage, battery, camera, display, price, and other specifications are used to calculate compatibility with customer preferences. This approach allows the system to recommend smartphones based on their characteristics even when limited interaction data is available.

2.2.8 Hybrid Recommendation System

A Hybrid Recommendation System combines two or more recommendation approaches to improve the quality and coverage of recommendations [22]. Combining different approaches can reduce the limitations of using a single recommendation technique.

In this project, content based recommendation is combined with collaborative filtering. Customer segmentation is also used as an additional behavioural signal, while the collaborative filtering component combines SVD and item based cosine similarity. This allows the system to consider smartphone characteristics, customer preferences, customer segments, and interaction behaviour.

2.2.9 Singular Value Decomposition (SVD)

Singular Value Decomposition (SVD) is a matrix factorisation technique that can be used in collaborative filtering to represent users and items in a lower dimensional latent space. It decomposes the user–item interaction matrix into factors that capture hidden relationships between users and items [23].

Matrix factorisation methods are useful for discovering latent relationships between users and items from interaction data. In this project, SVD is used to learn customer and smartphone relationships from the interaction data. Its predictions are combined with item based cosine similarity to form the collaborative filtering hybrid model.

2.2.10 Cosine Similarity

Cosine Similarity is a measure used to determine the similarity between two vectors based on the angle between them [24]. A higher cosine similarity indicates that the two vectors have more similar characteristics.

In this project, cosine similarity is used in the recommendation process to measure similarity between smartphones and to support item based collaborative filtering. It is also used to compare customer preferences with smartphone features in the content based recommendation process.

3. Methodology

The data sources used, the preprocessing techniques applied, the machine learning algorithms selected with their underlying concepts, and the evaluation metrics used to measure the performance of the trained models are described below:

3.1 Block Diagram of the System

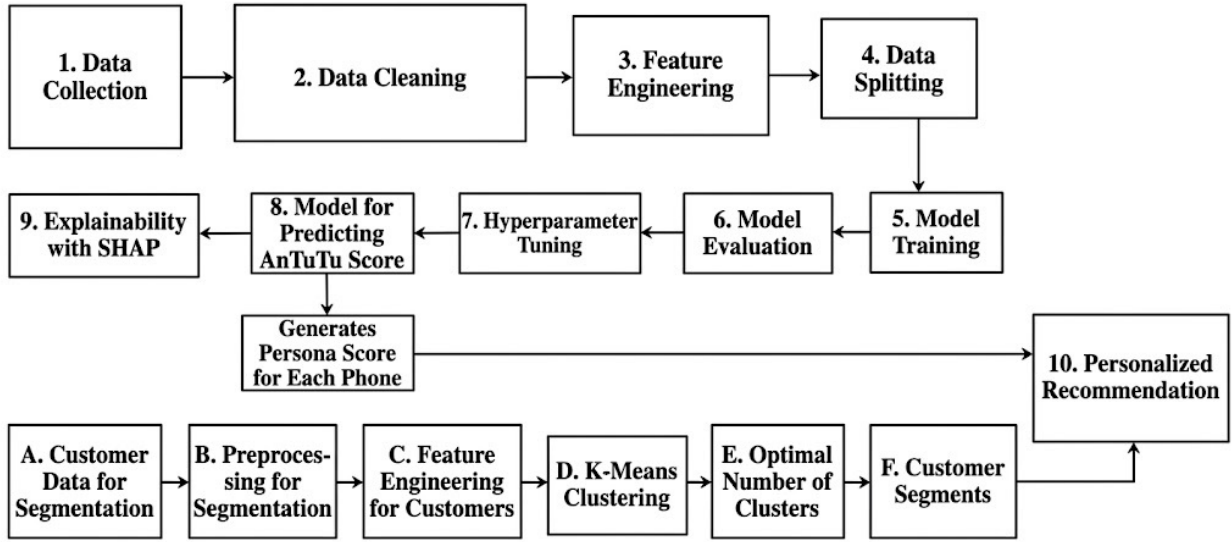


Figure 3.1: Block diagram of the System

The pipeline collects smartphone specifications, AnTuTu scores, and customer data. After preprocessing, XGBoost predicts smartphone performance, while K-Means identifies customer segments. SHAP explains the model predictions, and customer segments, content based recommendation, and collaborative filtering are combined to form the hybrid recommendation system

3.1.1 Data Collection

The proposed system requires three main categories of input data. The first is a dataset of mobile phone specifications, which describes the smartphones that can be recommended. The second is a dataset of customer profiles and transaction information, which is used to understand customer preferences and behaviour. The third is an interaction dataset

used by the collaborative filtering component to learn relationships between customers and smartphones. The datasets are stored in tabular form and are processed before being used by the machine learning and recommendation pipelines.

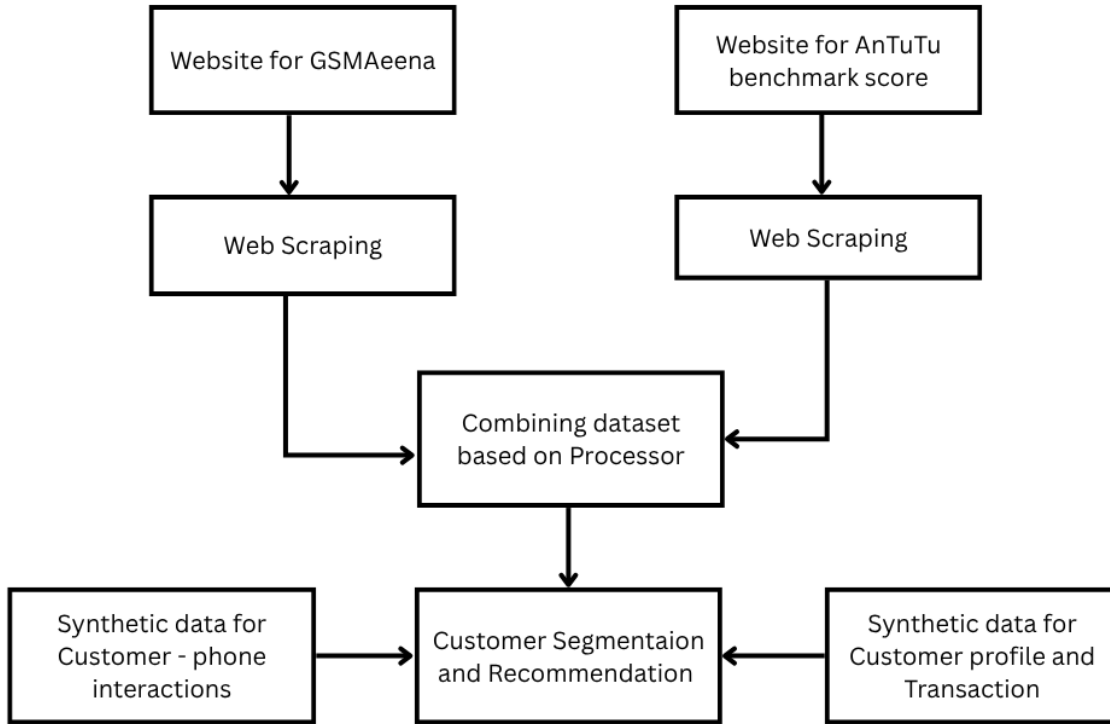


Figure 3.2: Data Collection and Integration Process

Phone Specifications

The phone specification dataset is collected from a public mobile phone database called GSMArena that lists the technical details of mobile phones released over the last several years. The source is selected because it provides a large collection of smartphone specifications that can be used by the recommendation system. For every phone, the source provides fields covering network technology, body dimensions, display, platform, memory, camera, connectivity, battery, and other features. The source also provides the model identifier and model image, which are useful for identifying and presenting smartphones in the web application.

Because the complete catalogue is large, the data collection process is divided into smaller batches. The collected batches are then combined into a single dataset, which forms the input to the phone preprocessing and feature engineering stages.

Benchmark Scores

A second public source is used to collect benchmark scores for smartphones. AnTuTu benchmark scores are used as an indication of overall smartphone performance. The collected benchmark data contains the model identifier and benchmark value. The benchmark data is merged with the smartphone specification dataset by matching cleaned versions of the model identifiers.

The merged dataset contains smartphone specifications together with the available benchmark score. Missing benchmark values are retained as missing rather than being used as training targets. Only records with measured benchmark values are used to train the XGBoost performance prediction model to avoid introducing artificially generated target values into the training process.

Customer Profile and Transaction Data

The customer side of the system uses a synthetic customer dataset designed to represent the type of information that may be available to a mobile retailer. The dataset contains customer profiles, preferences, purchasing information, spending behaviour, brand preferences, browsing information, wishlist information, and other behavioural attributes.

The original transaction records are processed and aggregated to create customer level profiles. These profiles are used for customer segmentation and to understand differences in customer preferences and purchasing behaviour. The use of synthetic data allows the project to demonstrate the complete customer segmentation and recommendation pipeline without using private customer information.

Customer-Phone Interaction Data

Collaborative filtering requires multiple interactions between customers and smartphones rather than only one aggregated record per customer. Therefore, a separate interaction dataset is prepared from the customer profiles and smartphone catalogue. A specification matching process is used to associate customers with smartphones that satisfy their relevant preferences and requirements.

Based on these relationships, an interaction log containing 86,195 records is generated for 4,557 customers and 1,787 smartphones. The interaction records include different types of customer activity such as views, searches, comparisons, wishlists, purchases, and ratings. Each interaction contains the customer identifier, smartphone model, interaction type, rating where available, and timestamp.

This interaction dataset provides the user–item information required for collaborative filtering. It also allows the system to represent customer behaviour over time instead of relying only on aggregated customer statistics.

3.1.2 Data cleaning and Preprocessing

The preprocessing stage transforms the collected data into clean and model-ready datasets. The processing is performed through separate but connected streams for smartphone data, customer profile data, and customer–phone interaction data. The smartphone pipeline prepares the features required for performance prediction and content based recommendation, while the customer pipeline prepares features for segmentation. The interaction pipeline prepares the user–item data required for collaborative filtering.

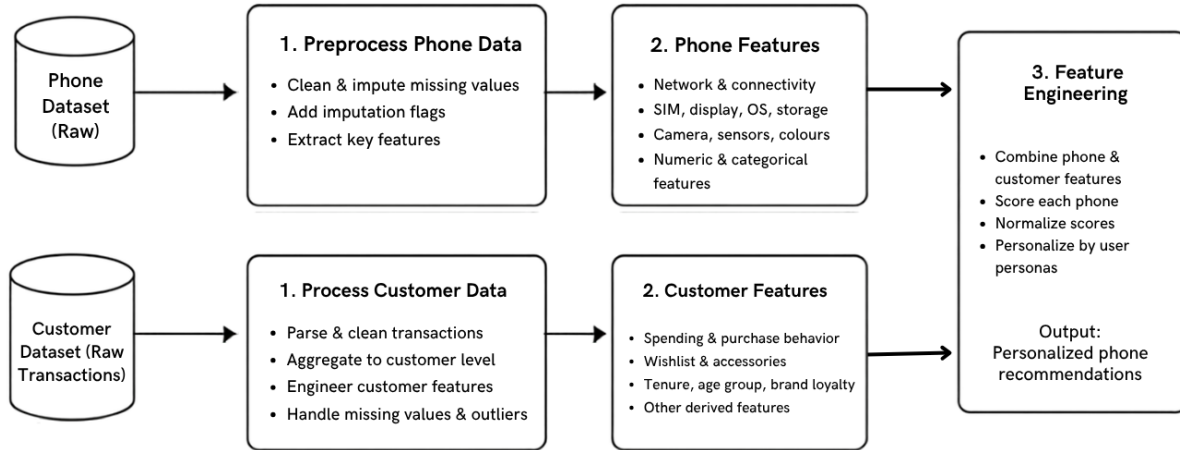


Figure 3.3: Data Processing Flow

Phone Dataset Preprocessing

The CPU, chipset, and GPU columns are converted from free text into structured features. CPU cores, clock speed, architecture, chipset details, GPU vendor, and GPU family are extracted. Missing chipset and GPU values are filled using similar devices based on CPU, chipset, brand, and release year.

Important numeric columns are filled using a three-level median imputation strategy. For selected fields, imputation flags are added to indicate whether the original value was missing.

The network, status, and dimensions columns are transformed into numeric features. Network generation and 5G support are identified, release year and availability are extracted, and device dimensions are used to calculate volume, aspect ratio, and density.

The SIM, display, operating system, and storage columns are converted into compact categorical and binary features. These include SIM type, display technologies, OS version, software support, storage family, and storage performance tier.

The camera, Wi-Fi, GPS, sensors, colours, and FM radio columns are also processed into structured features. Camera capabilities, wireless connectivity, available sensors, and colour count are extracted, while very rare features are removed.

Finally, all original free-text columns are dropped after feature extraction. The final dataset contains numeric, binary, and low-cardinality categorical features suitable for machine learning models and recommendation.

Customer Dataset Preprocessing

The customer preprocessing stage converts transaction-level records into a customer-level dataset. List-based columns such as browsing history, wishlist, accessories purchased, and exchange history are first parsed into structured data after correcting boolean values.

The transaction records are then grouped by customer to calculate summary statistics. These include purchase count, total and average spending, purchase frequency, ratings, recency, first and last purchase dates, preferred categories, wishlist and accessory information, exchange count, and brand loyalty.

Additional customer features are created after aggregation. These include customer tenure, wishlist conversion rate, average accessory price, price sensitivity, and age group.

Finally, the aggregated customer dataset is checked for missing values and outliers. Derived columns are filled using median values when necessary, while outlier flags are added for spending related features without removing customer records.

Customer Segmentation Preprocessing

A separate preprocessing stage is applied to the customer features used for segmentation. Numerical customer features are standardised so that features with different ranges do not disproportionately influence the clustering process. Categorical customer features are converted into numerical representations using one hot encoding.

The resulting customer feature matrix contains behavioural, preference, spending, and categorical features. The final matrix contains 4,557 customers and 61 engineered features. This matrix is used as the input to the K-Means clustering algorithm.

The number of clusters is evaluated using clustering validation techniques, including the

elbow method and silhouette score. Four clusters are selected based on the clustering evaluation, producing the customer groups Hardcore Gamer, Mainstream Mid Range Shopper, Premium Flagship Shopper, and Budget Buyer.

The trained K-Means model is saved so that the same preprocessing and clustering process can be applied to new customers. The assigned cluster information is also used by the recommendation system as an additional customer preference signal and for cold start recommendations.

Interaction Dataset Preprocessing

The interaction dataset is prepared specifically for the collaborative filtering component. Customer identifiers and smartphone model names are standardised to ensure that the same customer and smartphone are represented consistently throughout the dataset. Duplicate and invalid interactions are removed where necessary.

Each interaction contains a customer identifier, smartphone model, interaction type, rating where available, and timestamp. Different interaction types represent different levels of customer interest, including viewing, searching, comparing, adding a smartphone to a wishlist, purchasing, and rating.

The interaction information is converted into a user-item representation that can be processed by collaborative filtering algorithms. The data is divided according to time so that earlier interactions are used for training and later interactions are used for evaluation. This prevents future interactions from being used to predict past behaviour and provides a more realistic evaluation of the recommendation system.

A separate cold start holdout is also maintained for evaluating recommendation behaviour for customers and smartphones with limited historical interaction information.

3.1.3 Feature Engineering

The cleaned phone dataset undergoes a second stage of feature engineering to create nine user-oriented scores on a 0–100 scale. These scores represent gaming, camera, battery, display, software, security, connectivity, portability, and storage performance using weighted combinations of relevant hardware and software features.

An overall phone score is calculated as the weighted sum of the nine dimension scores. A value score is also computed by comparing the overall score with the phone’s price, helping identify devices that offer better value for money.

To ensure consistent scoring, numeric features are normalised using clipping and logarithmic

normalisation methods. The same normalisation parameters are reused during prediction to maintain consistency.

Several user personas such as Gamer, Camera Lover, Battery Focused, All-rounder, and Business User are defined using different weight combinations. A custom persona is also supported, allowing users to assign their own importance ratings to each dimension for personalised recommendations.

For customer segmentation, additional features are derived from customer behaviour, preferences, spending, and purchasing patterns. These features provide the basis for identifying groups of customers with similar characteristics.

3.1.4 Scaling, Outlier Handling, and Train Test Split

The XGBoost regressor used in this project is a tree based model, so a separate scaling step is not required for the raw input features. Outliers are handled through percentile clipping used in the feature normalisation process, which keeps the engineered scores within a suitable range.

Categorical features are processed consistently between training and prediction so that the model receives the same feature representation during inference. The regression dataset is divided into training and test sets using an 80/20 split with a fixed random seed. Five fold cross-validation is also performed using the training data to evaluate model performance more reliably.

The XGBoost model is trained only on records where the AnTuTu benchmark score is available as a measured value. Records with imputed or unavailable benchmark scores are not used as training targets. This ensures that the model learns the relationship between actual smartphone specifications and measured performance rather than learning from artificially generated target values.

3.1.5 Segmentation Model

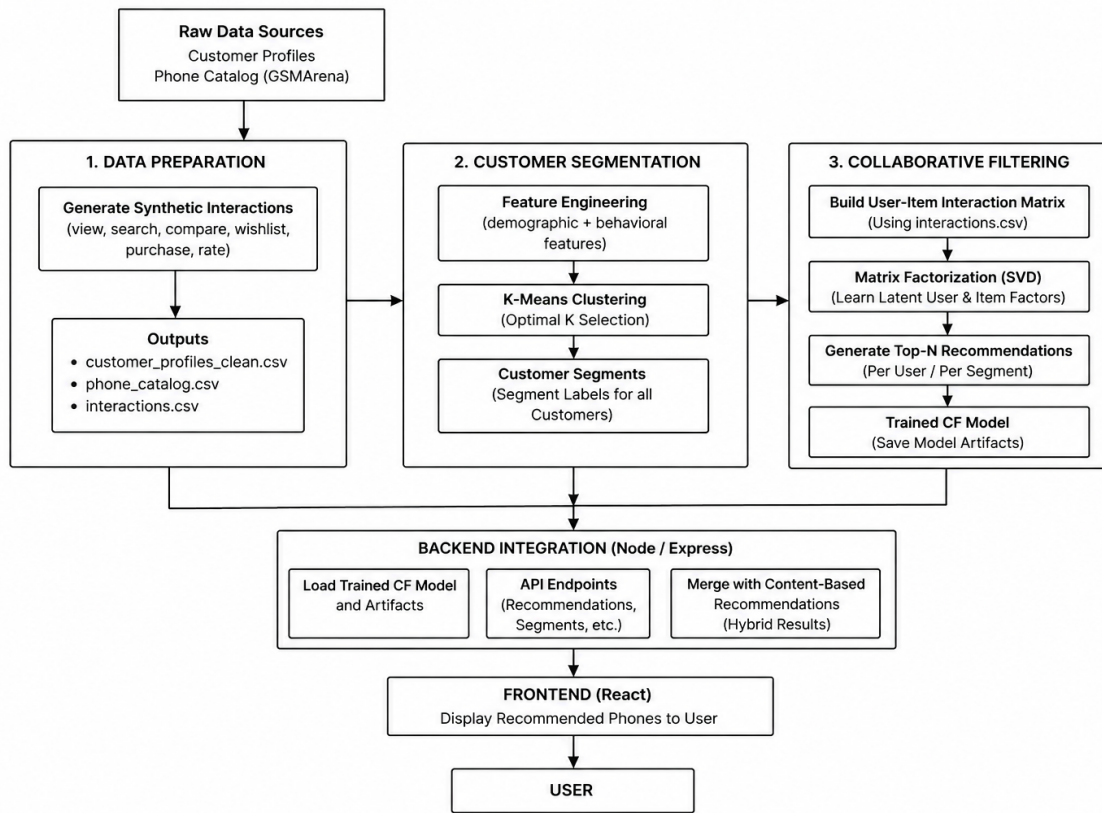


Figure 3.4: Customer Segmentation Flow

The prepared customer feature matrix is used to train the K-Means clustering model. Numerical features are standardised and categorical features are encoded before clustering. Several values of K are evaluated to determine an appropriate number of customer groups.

The silhouette score is used to compare the quality of the resulting clusters, while the elbow method is used as an additional measure for selecting the number of clusters. The final model uses $K = 4$, resulting in four customer segments. The cluster assignments and customer segment names are stored with the customer profiles and are later used by the recommendation pipeline.

3.1.6 Recommendation Engine

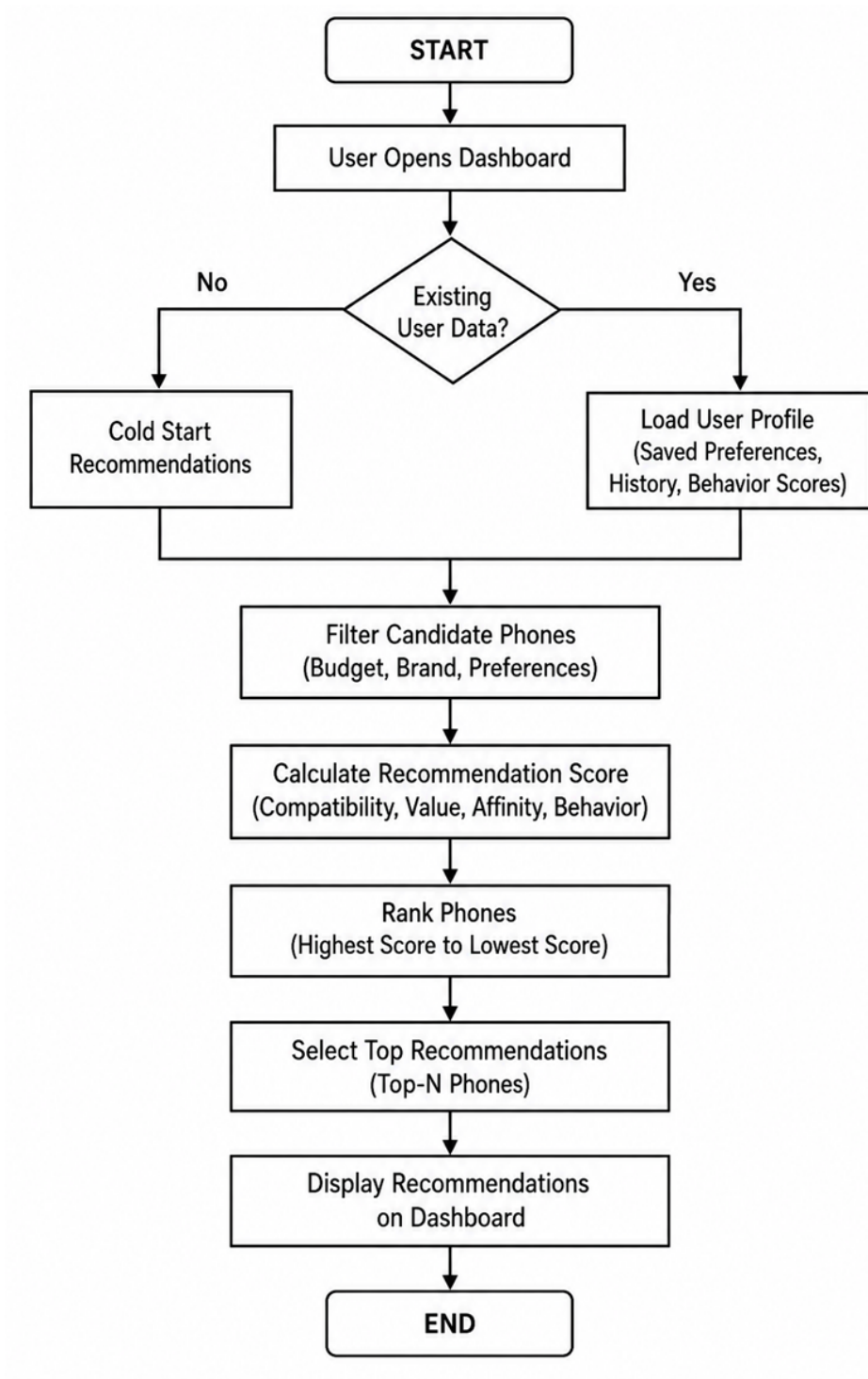


Figure 3.5: Recommendation Engine Workflow

The recommendation system uses both smartphone content and customer behaviour. The content based component uses smartphone specifications and engineered phone scores to

measure the compatibility between a customer’s preferences and available smartphones. The collaborative filtering component uses the prepared customer–phone interaction data to learn behavioural relationships.

For collaborative filtering, item based cosine similarity, Singular Value Decomposition (SVD), and Implicit Alternating Least Squares (ALS) are evaluated. The SVD and item cosine similarity combination provides the best ranking performance and is selected as the collaborative filtering model.

The final recommendation process combines the information from the content based and collaborative filtering components. Customer segmentation provides an additional signal for recommendation and supports the cold start strategy. For customers with insufficient interaction history, recommendations are generated using cluster popularity followed by province, district, and global popularity before applying the relevant smartphone content and budget constraints.

3.2 Methods and Algorithms

The machine learning stack has two parts. The first is a phone level regression model that scores a phone on nine dimensions and predicts the benchmark value for new phones. The second is a customer level clustering model that groups customers into segments based on their purchase history. A weighted scoring ranker sits on top of the phone model and produces the recommendations.

3.2.1 XGBoost Algorithm

The phone prediction model uses an Extreme Gradient Boosting (XGBoost) regressor to estimate the benchmark score from the processed phone features. The target benchmark score is transformed using $\log(1 + x)$ to reduce skewness and improve prediction accuracy, especially for high-end flagship devices. The model is trained using the fixed hyperparameters listed in Table 6.3.

Mathematical formulation: Extreme gradient boosting is a regularised ensemble of K additive decision trees, with the objective

$$\mathcal{L}(\theta) = \sum_{i=1}^n l(y_i, \hat{g}_i) + \sum_{k=1}^K \Omega(f_k) \quad (3.1)$$

where l is the squared error loss on the log transformed target, f_k is the k -th tree, and Ω is a regularisation term. The regularisation used in extreme gradient boosting is

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2 \quad (3.2)$$

where T is the number of leaves in the tree, w_j is the score assigned to leaf j , γ is the complexity penalty per leaf, and λ is the L2 weight decay.

At each boosting round the model fits a new tree to the negative gradient of the loss with respect to the current prediction, and uses the second order Taylor expansion of the loss to pick the best split. For a candidate split that sends I_L samples to the left and I_R samples to the right, the gain is

$$\mathcal{G}_{\text{split}} = \frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda} \right] - \gamma \quad (3.3)$$

where G and H are the sums of the first and second order gradients in the node. The split with the largest gain is chosen.

Reason for using extreme gradient boosting for this project: The phone data is mostly tabular, mostly numeric, and contains both continuous values and categorical values. The model handles both kinds of feature without one hot encoding, captures non-linear interactions between features, and is resistant to monotone feature transforms so the logarithmic target and the clipped scores do not need extra preprocessing. The cross-validated coefficient of determination and the test coefficient of determination confirm that the regularised ensemble generalises well beyond the training rows.

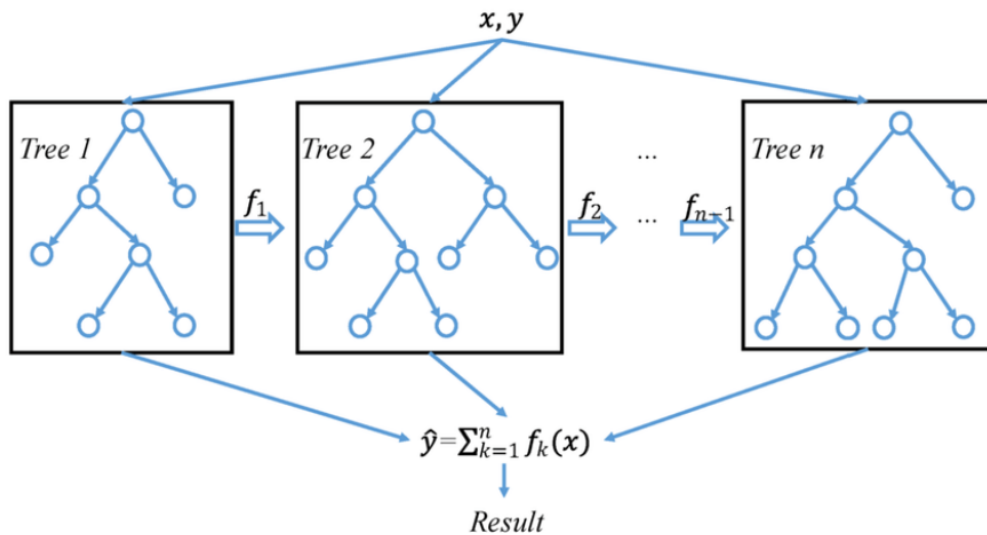


Figure 3.6: XGBoost Ensemble of Decision Trees [1]

3.2.2 K-Means Clustering for Customer Segmentation

Customer segmentation uses the K-Means algorithm on a customer by feature matrix. Twenty numeric features and six categorical features are fed into a column transformer that imputes missing numerics with the median, standardises them with a standard scaler, imputes missing categoricals with the most frequent value, and one hot encodes them with a one hot encoder that drops the first category and ignores unknown categories. The resulting matrix has 64 columns. K-Means is fit with a large number of initialisations and a fixed random seed so the result is reproducible.

Mathematical formulation: K-Means partitions N points into K clusters by minimising the within cluster sum of squared distances,

$$J = \sum_{k=1}^K \sum_{x \in C_k} \|x - \mu_k\|^2 \quad (3.4)$$

where μ_k is the centroid of cluster C_k . Lloyd’s algorithm alternates between assigning each point to the nearest centroid and recomputing the centroid of each cluster, and converges to a local minimum of J .

Number of clusters. The number of clusters is selected by running K-Means for a range of values of k and recording four metrics: the inertia, the silhouette score, the Davies-Bouldin index, and the Calinski-Harabasz index. The chosen k is the one with the highest Calinski-Harabasz score, subject to a minimum cluster share so that no segment is too small to be useful in a marketing campaign, and to a minimum number of clusters. Agglomerative clustering with Ward linkage and density based spatial clustering are also run on the same data as a cross check, and the agreement of the two methods on the broad shape of the segmentation is used as additional evidence for the choice of k .

Reason to use K-Means for this project: The customer data is mostly continuous and the K-Means assumption of roughly spherical clusters is reasonable after the standardisation. K-Means is also the cheapest clustering algorithm to refit when new transaction rows arrive, and the trained model is small enough to be persisted as a single file.

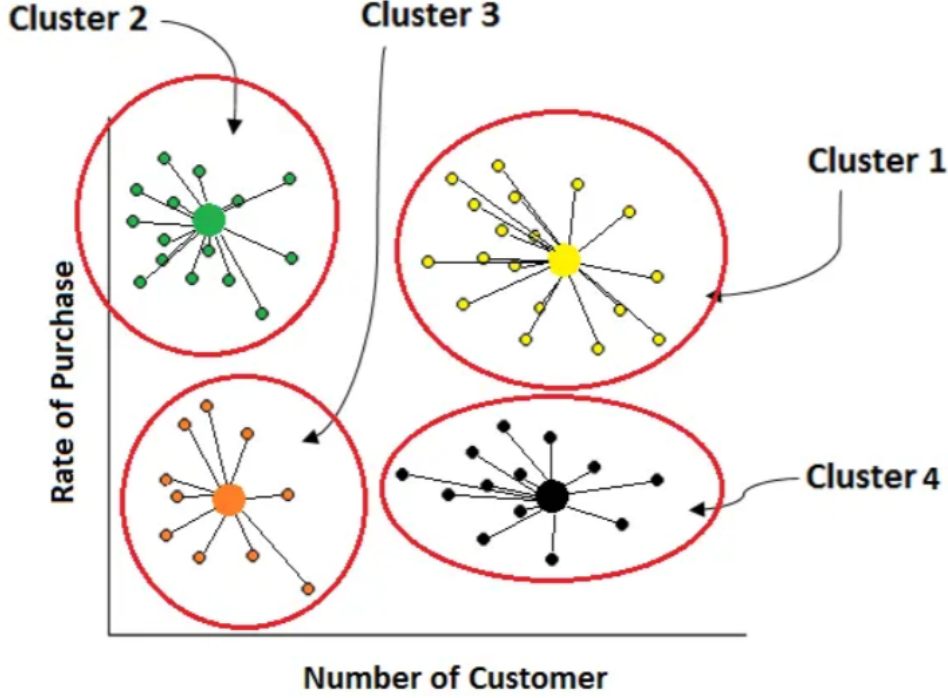


Figure 3.7: K-Means Clustering of Customers [2]

3.3 Evaluation Metrics

The project uses three families of evaluation metrics. The first family is for the regressor that predicts the benchmark score. The second family is for the clustering algorithm that produces the customer segments. The third family is for the recommendation engine, which is the ranker that sits on top of the two models.

3.3.1 Metrics for the Regressor

Coefficient of determination (R^2). The coefficient of determination measures the fraction of variance in the target that the model explains. It is defined as

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2} \quad (3.5)$$

where y_i is the true log transformed benchmark score, $\hat{y}_i$ is the prediction, and $\bar{y}$ is the mean of the true values. A value of 1 means the model explains all of the variance, a value of 0 means it does no better than predicting the mean, and a negative value means it does worse. In this project the coefficient of determination is computed both on the 80% training set and on the 20% held out test set, and also averaged over the 5 folds of a cross-validation, to check that the model is not overfitting. The metric is sensitive to the variance of the target,

so it is not directly comparable across datasets, but it is a good internal check.

Mean absolute error (MAE). The mean absolute error measures the average absolute deviation of the predictions from the true values, in the same units as the target.

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (3.6)$$

In this project the mean absolute error is reported in log space because the model is trained on the logarithm of one plus the benchmark score. The metric is robust to outliers, which matters because the benchmark distribution has a long flagship tail.

Mean squared error (MSE). The mean squared error measures the average of the squared differences between the predictions and the true values. It is defined as

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (3.7)$$

where y_i is the true log transformed benchmark score and $\hat{y}_i$ is the prediction. Because the errors are squared before averaging, the metric penalises larger errors more heavily than smaller ones, which makes it sensitive to outliers. In this project the mean squared error is reported in log space to remain consistent with the log transformed target on which the model is trained.

Root mean squared error (RMSE). The root mean squared error is the square root of the mean squared error and is expressed in the same units as the target. It is defined as

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (3.8)$$

where y_i is the true log transformed benchmark score and $\hat{y}_i$ is the prediction. Taking the square root restores the original units of the target, which makes the root mean squared error directly comparable to the mean absolute error for the same target. In this project the root mean squared error is reported in log space so that the magnitude of the typical error can be compared with the mean absolute error.

Mean absolute percentage error (MAPE). The mean absolute percentage error expresses the average absolute deviation of the predictions from the true values as a percentage of the true value. It is defined as

$$\text{MAPE} = \frac{1}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right| \times 100\% \quad (3.9)$$

where y_i is the true value and $\hat{y}_i$ is the prediction. The metric is scale independent, which makes it convenient for reporting the average relative error, but it is undefined when the true value is zero and it is asymmetric in the sense that it tends to penalise under predictions more heavily than over predictions. In this project the mean absolute percentage error is computed on the original benchmark scale to provide an intuitive summary of the average relative deviation of the predicted score from the true score.

Root mean squared logarithmic error (RMSLE). The root mean squared logarithmic error is the square root of the mean of the squared logarithmic differences between the predictions and the true values. It is defined as

$$\text{RMSLE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (\log(1 + y_i) - \log(1 + \hat{y}_i))^2} \quad (3.10)$$

where y_i is the true value and $\hat{y}_i$ is the prediction. The log transformation inside the formula compresses large values and expands small values, which makes the metric well suited to targets with a long right tail such as the benchmark distribution used in this project. The root mean squared logarithmic error is reported on the original benchmark scale, which provides a complementary view of the error to the mean absolute error and the root mean squared error that are both reported in log space.

Adjusted coefficient of determination (Adjusted R^2). The adjusted coefficient of determination adjusts the coefficient of determination for the number of features used by the model, so that adding a feature that does not improve the fit does not artificially increase the metric. It is defined as

$$\bar{R}^2 = 1 - (1 - R^2) \cdot \frac{n - 1}{n - p - 1} \quad (3.11)$$

where R^2 is the coefficient of determination, n is the number of samples and p is the number of features. The adjusted R^2 is always lower than or equal to R^2 and only increases when a new feature improves the fit more than would be expected by chance. In this project the adjusted R^2 is reported alongside R^2 to confirm that the predictive power of the model is not driven by the large number of input features alone.

K fold cross validation. A 5 fold cross-validation is run with a fixed seed. For each fold the model is refit on the other four folds and scored on the held out fold, and the mean and standard deviation of the coefficient of determination across folds are reported. The cross-validated scores are the most trustworthy number for predicting how the model will behave on phones that arrive after the training set was frozen.

3.3.2 Metrics for the Segmentation

Silhouette score. For a point i with cluster assignment C_i , the silhouette score is

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}} \quad (3.12)$$

where $a(i)$ is the mean distance from i to the other points in the same cluster and $b(i)$ is the mean distance from i to the points in the nearest other cluster. The reported silhouette is the mean of $s(i)$ over all points. It ranges from -1 to 1 , with higher values meaning better defined clusters. The metric is useful for picking k because it is monotonic in cluster separation.

3.3.3 Metrics for the Recommendation Engine

The recommendation engine is a ranker, so the relevant metrics are ranking metrics rather than regression metrics. Three metrics are planned for the final evaluation of the engine.

Mean reciprocal rank (MRR). For a list of N users, the mean reciprocal rank is

$$\text{MRR} = \frac{1}{N} \sum_{u=1}^N \frac{1}{\text{rank}_u} \quad (3.13)$$

where rank_u is the position of the first relevant phone in the ranked list for user u . A higher value means the relevant phone tends to be returned closer to the top. The metric is appropriate when there is at most one ground truth relevant phone per user, which is the case in this project because each user typically buys one phone at a time.

Normalised discounted cumulative gain at K (NDCG@K). NDCG@K rewards rankings where relevant items appear near the top, with a logarithmic discount for lower positions. It is

$$\text{NDCG@K} = \frac{\text{DCG@K}}{\text{IDCG@K}}, \quad \text{DCG@K} = \sum_{i=1}^K \frac{\text{rel}_i}{\log_2(i+1)} \quad (3.14)$$

where rel_i is the relevance of the item at position i and IDCG is the discounted cumulative gain of the ideal ordering. NDCG@K is in $[0, 1]$, with 1 being a perfect ranking. The metric is appropriate when the relevance is graded rather than binary, which fits the case where a user might consider several of the returned phones.

4. Experimental Setup

The hardware and software environment used for data collection, data preprocessing, customer segmentation, model training, recommendation model development, evaluation, and web application development are described below.

4.1 Hardware Configuration

The proposed system was developed and tested on a personal workstation. The hardware configuration used during development is as follows:

- **Processor:** Intel Core i5 13th Generation
- **RAM:** 16 GB
- **Storage:** 512 GB SSD
- **GPU:** Not used for model training.

The machine learning and recommendation models were primarily executed using the CPU. The GPU was not specifically used for training the XGBoost, K-Means, SVD, or cosine similarity based models. The available GPU was mainly part of the development workstation and was not required by the implemented machine learning pipeline.

4.2 Software and Tools

The system was developed using a combination of Python based machine learning tools and web development technologies. The main software and libraries used are listed below:

- **Python 3.10** for data preprocessing, feature engineering, machine learning, and recommendation model development.
- **Pandas** and **NumPy** for data manipulation, numerical computation, and dataset processing.
- **Scikit-learn** for preprocessing, K-Means clustering, cosine similarity, model evaluation, and other machine learning operations.
- **XGBoost** for predicting smartphone performance from smartphone specifications.
- **SHAP** for interpreting the predictions of the XGBoost model and analysing important

smartphone features.

- **Selenium** and **BeautifulSoup** for collecting smartphone specifications and benchmark data from web sources.
- **Matplotlib** and **Seaborn** for generating graphs and visualising model and segmentation results.
- **Python SVD and collaborative filtering libraries** for learning customer–smartphone interaction patterns and generating personalised recommendations.
- **Node.js** and **Express.js** for developing the backend services and REST APIs.
- **React.js** for developing the web based user interface.
- **PostgreSQL** for storing user, customer, phone, transaction, and application data.
- **Prisma ORM** for database schema management and interaction between the backend and PostgreSQL.
- **FastAPI** for serving the machine learning and recommendation models through APIs.
- **Passport.js** and **bcrypt** for user authentication and secure password handling.
- **Docker and Docker Compose** for containerising and managing the backend, database, and machine learning services.

5. System Design

This chapter describes the requirement specification and the system diagrams that graphically represent the system, its components, and operations.

5.1 Requirement Specification

5.1.1 Functional Requirements

1. The system shall allow customers to register, log in, and verify their account through email-based OTP.
2. The system shall allow customers to enter their preferences (budget, brand, performance priority, and storage) for generating recommendations.
3. The system shall generate personalised mobile phone recommendations based on customer preferences and segmentation.
4. The system shall allow sales staff to enter customer details and view the recommended phones with their camera tier and tech tier.
5. The system shall allow the administrator to manage users, assign roles, and monitor system activity.
6. The system shall explain predictions using SHAP feature importance.

5.1.2 Non-Functional Requirements

1. **Performance:** The system shall generate recommendations within a few seconds for a typical user interaction.
2. **Security:** Passwords shall be stored using bcrypt hashing, and OTP codes shall expire after a limited time.
3. **Usability:** The interface shall be responsive and intuitive for customers, sales staff, and administrators.
4. **Scalability:** The backend shall be designed to scale horizontally as the number of users grows.
5. **Maintainability:** The codebase shall follow modular design patterns to enable future

feature additions.

5.2 System Architecture

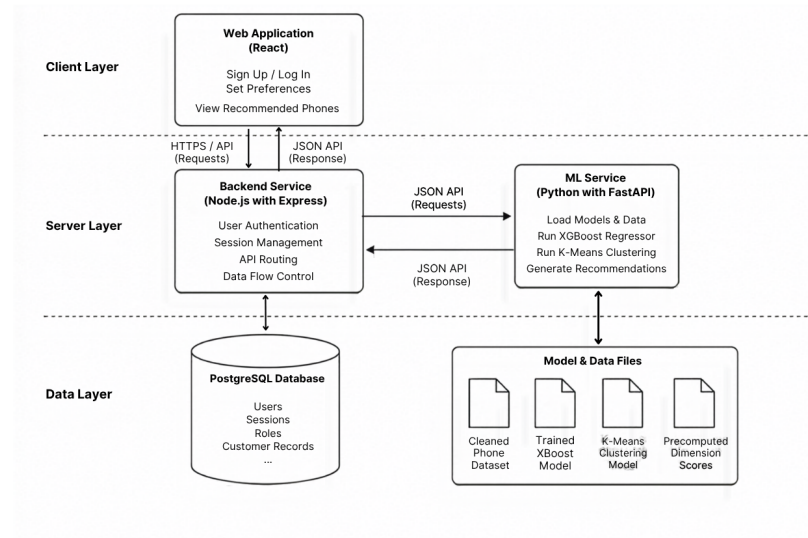


Figure 5.1: System Architecture of the Mobile Recommendation System

The system architecture illustrates the layered design of the project, comprising the frontend, backend, database, and machine learning model layers. The frontend is built using React.js and communicates with the backend through REST API endpoints. The backend uses Node.js and Express.js and applies business logic, while the database layer uses PostgreSQL through Prisma ORM. The machine learning layer encapsulates the trained XGBoost model and the K-Means clustering pipeline.

5.3 Use Case Diagram

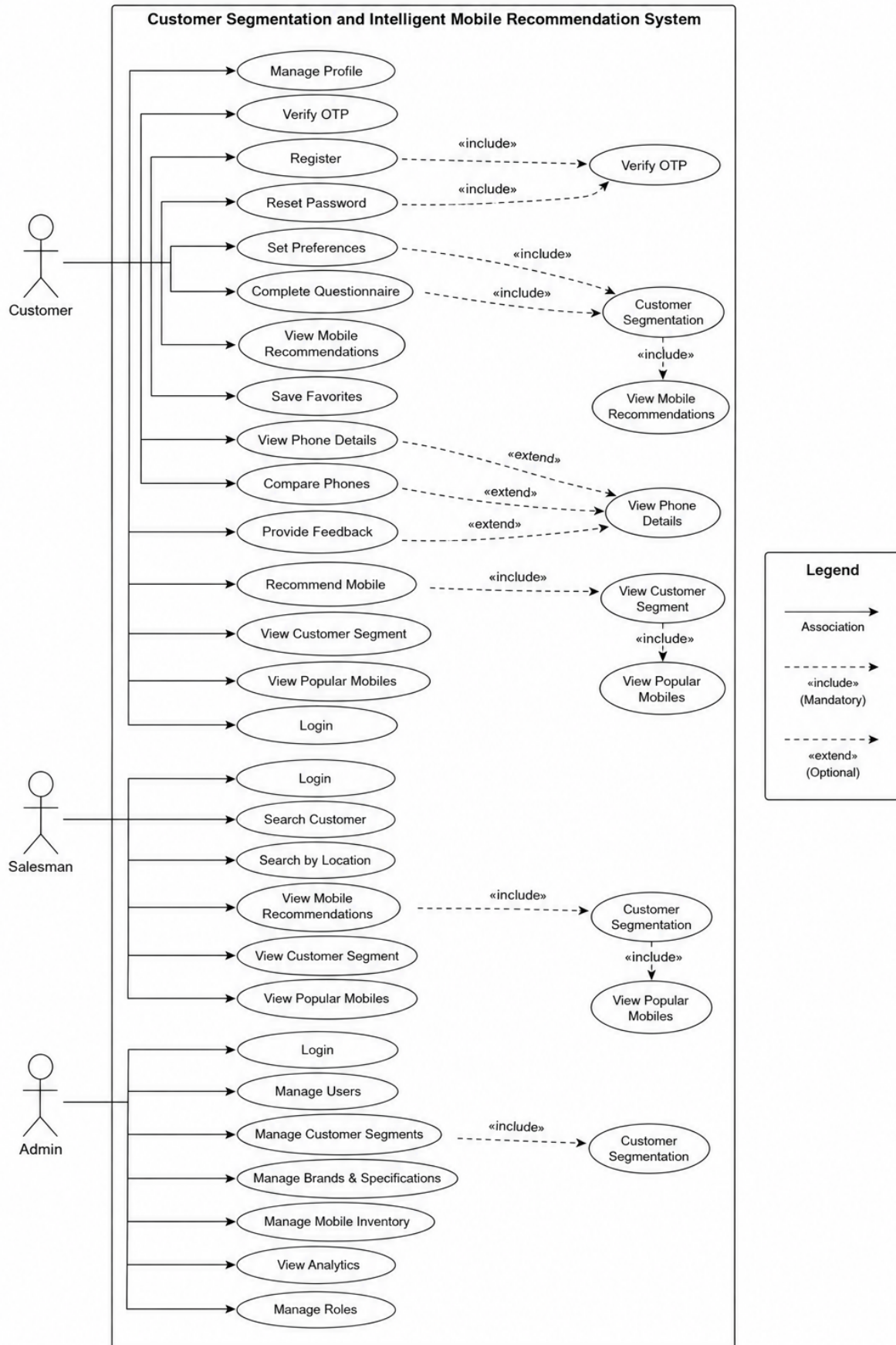


Figure 5.2: Use Case Diagram of the Mobile Recommendation System

The use case diagram depicts the three actor roles in the system, namely customer, salesman, and administrator, and the corresponding use cases available to each. The customer can register, log in, set preferences, view recommendations, and view details. The salesman can log in, enter customer details, and view phone recommendations. The administrator can log in, manage users, assign roles, and monitor the system.

5.4 Data Flow Diagram

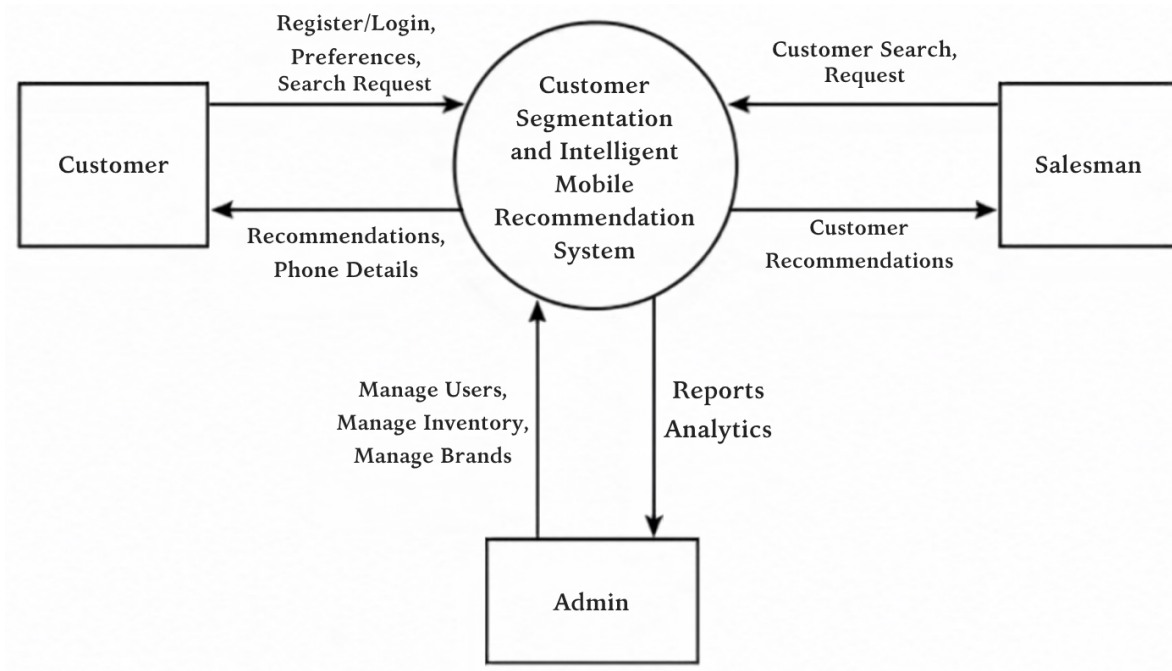


Figure 5.3: Level 0 Data Flow Diagram

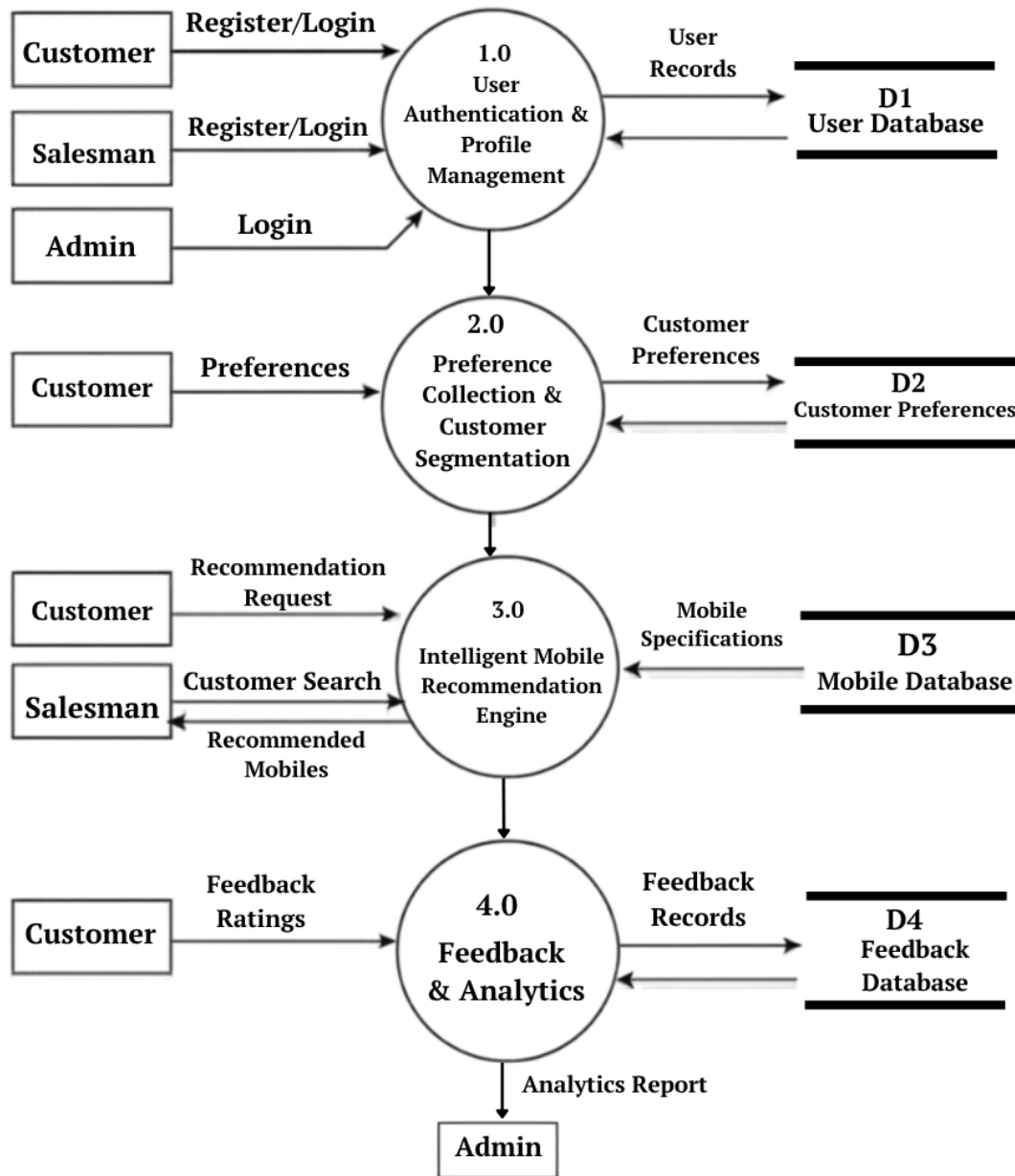


Figure 5.4: Level 1 Data Flow Diagram

The data flow diagrams describe how information moves between the customer, the system, and the database. At Level 0 the system is shown as a single process that interacts with an external customer and an external database. At Level 1 the internal processes are expanded to show the request handling, machine learning model invocation, and database storage operations.

5.5 Iterative Development Model

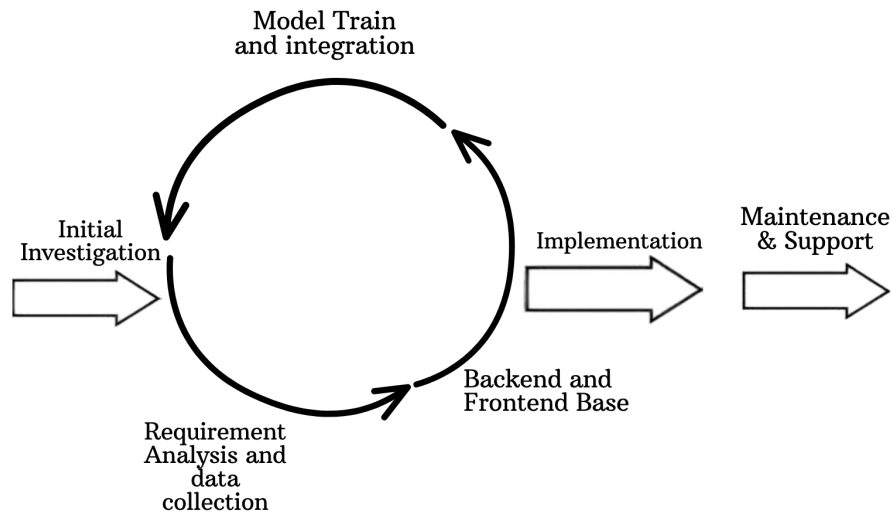


Figure 5.5: Iterative Development Model

The project uses an iterative development model in which each iteration refines the previous output. The iterations include requirements gathering, system design, implementation, testing, and review. The model supports continuous improvement and accommodates evolving requirements.

6. Results & Discussion

This chapter presents the results of the implemented system, the performance of the trained models, and a discussion of the outcome. The results are supported by tables and figures that summarise the experimental findings.

6.1 Experiment Setup

The phone specification dataset and the AnTuTu benchmark scores were collected through web scraping. The customer transaction dataset was synthesised to simulate realistic retail data. The collected data was preprocessed and split into training and test sets in an 80/20 ratio with a fixed random seed. The XGBoost regressor was trained on the log-transformed target and evaluated using 5-fold cross-validation.

6.2 Data Preparation Results

The phone specification dataset and AnTuTu benchmark dataset were successfully collected through web scraping by dividing it into 20 batches and preprocessed. The phone specification and benchmark datasets were merged, while missing values and duplicate records were handled. Feature engineering was performed to prepare the data for machine learning.

Table 6.1: Phone Dataset Before Processing

Attribute	Value
Total rows (raw combined batches)	12,849
Total columns (raw)	68
Columns with >55% missing values	12
Exact duplicate rows	421
Rows with missing CPU, Chipset and GPU	1,219
Missing rate of AnTuTu_Score	52.5%
Missing rate of GeekBench_Score	46.1%
Missing rate of Chipset	89.6%
Missing rate of Price_EUR	27.2%
Missing rate of Announced_Year	1.1%

Table 6.1 shows the state of the raw phone dataset immediately after combining the 20

GSMArena scrape batches, before any cleaning or imputation was applied.

Table 6.2: Phone Dataset After Processing

Attribute	Value
Phone specification dataset	8,500 phones
Phone features after engineering	152 features
Customer dataset	4,557 customers
Cities covered	90
Exact duplicates removed	421 rows
Columns dropped (>55% missing)	12
Brand names corrected	596 rows
Missing rate of AnTuTu_Score	0.0%
Missing rate of Chipset	0.0%
Missing rate of Price_EUR	0.0%

Table 6.2 summarises the dataset after all preprocessing, cleaning, and feature engineering steps were completed.

6.3 Machine Learning Results

An XGBoost regression model was trained to predict the AnTuTu performance score from mobile phone specifications. The model achieved satisfactory prediction performance on the test dataset.

Table 6.3: XGBoost Model Hyperparameters

Hyperparameter	Value
enable_categorical	True
tree_method	hist
max_depth	10
min_child_weight	5
gamma	0.5
n_estimators	800
learning_rate	0.01
subsample	0.7
colsample_bytree	0.7
reg_alpha	1.0
reg_lambda	3.0

The XGBoost model was trained using a set of carefully selected hyperparameters to achieve better predictive performance and generalisation. Table 6.3 presents the final hyperparameter values used in the model.

Table 6.4: XGBoost Training and Testing Performance Metrics

Metric	Training	Testing
MAE	0.1416	0.1795
MSE	0.0507	0.0866
RMSE	0.2253	0.2944
MAPE	1.08%	1.37%
RMSLE	0.0160	0.0210
R^2 Score	0.9241	0.8819
Adjusted R^2	0.9225	0.8707

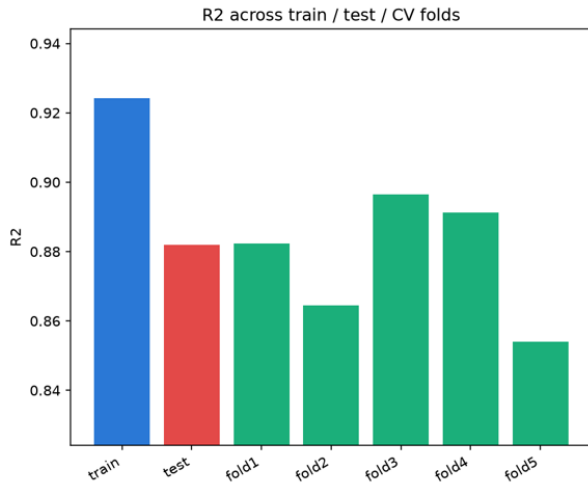
The performance of the XGBoost model was evaluated on both the training and testing datasets using standard regression metrics. Table 6.4 compares the training and testing performance, demonstrating the model’s ability to generalise well on unseen data.

Table 6.5: 5-Fold Cross-Validation Results

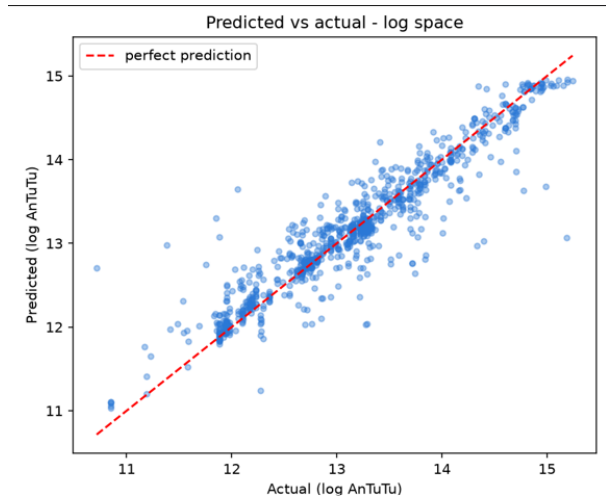
Metric	Value
Fold 1 R^2	0.8824
Fold 2 R^2	0.8645
Fold 3 R^2	0.8965
Fold 4 R^2	0.8912
Fold 5 R^2	0.8541
Mean R^2	0.8777
Standard Deviation	0.0160

To evaluate the stability and generalisation ability of the XGBoost model, 5-fold cross-validation was performed. The R-square scores for each fold, along with the mean and standard deviation, are presented in Table 6.5.

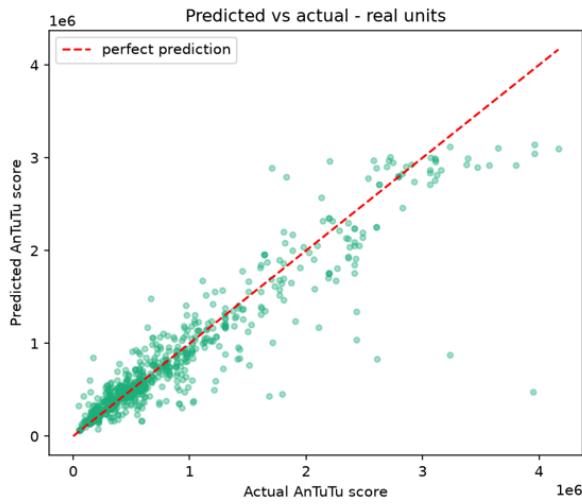
The obtained results indicate that the model predicts mobile phone performance with good accuracy and generalises well on unseen data.



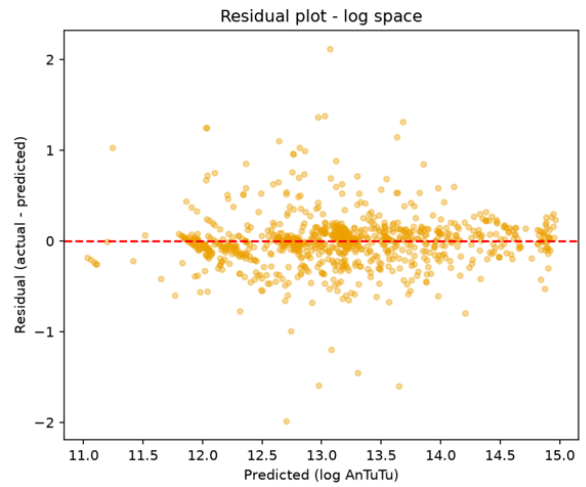
R^2 Score Validation Plot (Bar Graph)



Predicted vs. Actual Plot for Log AnTuTu



Predicted vs. Actual Plot for Real Units AnTuTu



Predicted vs Actual Residual plot

Figure 6.1: Visualizations of XGBoost Regressor Performance on the Test Set

To better understand the model's performance, several visualisations were generated from the trained XGBoost regressor. The predicted versus actual score plot shows that most predictions lie close to the diagonal line, indicating good accuracy. The error distribution is centred around zero, suggesting minimal prediction bias. The feature importance plot highlights the phone specifications that contribute most to the predicted AnTuTu score. Together, these visualisations support the strong performance metrics reported in the table.

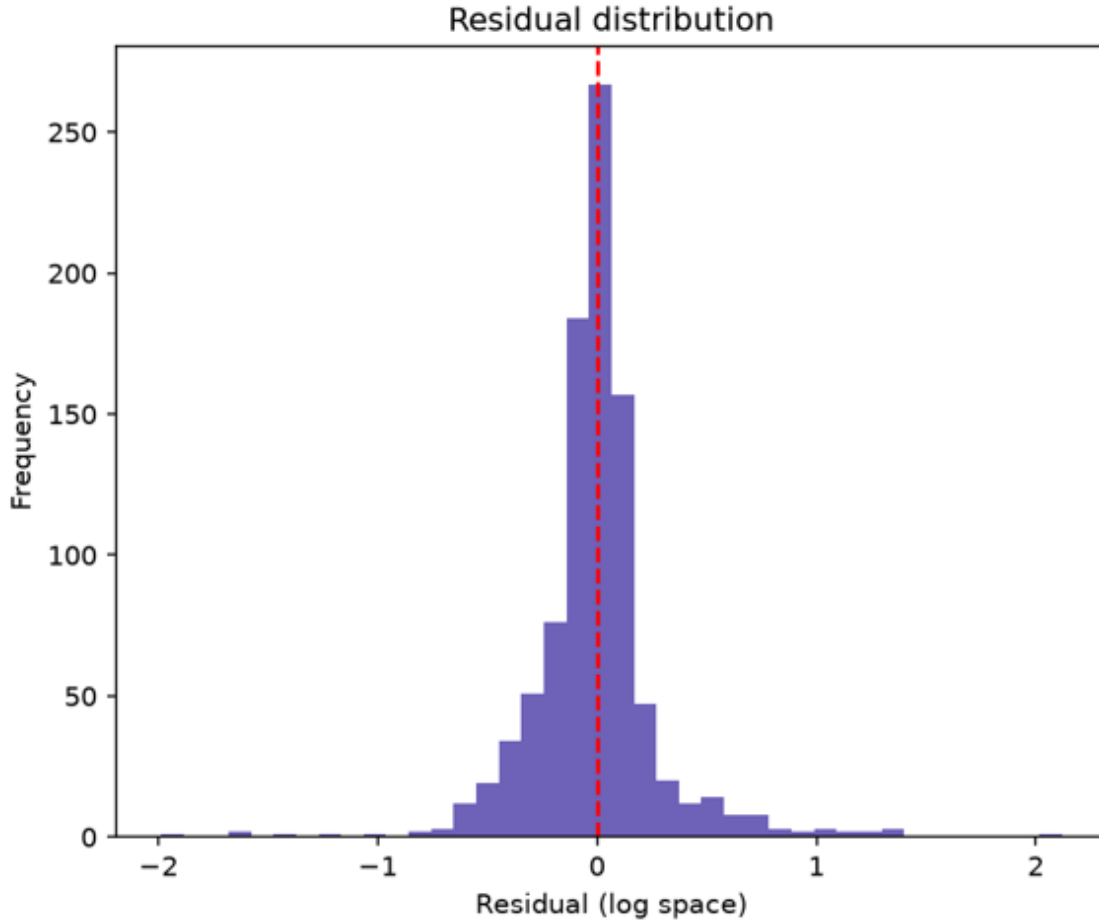


Figure 6.2: SHAP Feature Importance Summary Plot

The SHAP summary plot highlights the most influential phone features driving the predictions of the XGBoost model.

6.4 Recommendation System Output

The recommendation system was successfully implemented. The current system is capable of predicting the AnTuTu performance score of a phone, generating personalised phone recommendations, ranking recommended phones based on user preferences, and explaining prediction results using SHAP feature importance.

6.5 Software Development Approach

The web application was developed using an iterative development model. Each iteration added a new feature or refined an existing one based on testing and feedback. The backend was first implemented with RESTful endpoints, the database schema, and the authentication flow. The frontend was then connected through API integration, and the machine learning models were deployed through the backend.

6.6 Model Deployment

The trained XGBoost model and K-Means pipeline are serialised to disk as Python pickle files and loaded by the backend at startup. The Python machine learning service is invoked through a thin Node.js wrapper that calls a Python interpreter script for inference. This avoids embedding Python in the Node.js process while keeping the inference latency low.

6.7 User Interface and Backend

The backend has been developed using Node.js, Express.js, PostgreSQL, and Prisma ORM. Authentication features including user registration, login, OTP verification, password reset, and session management have been completed.

The frontend has been developed using React and currently includes: User Login Page, Registration Page, OTP Verification Page, Password Reset Page, and the Dashboard User Interface.

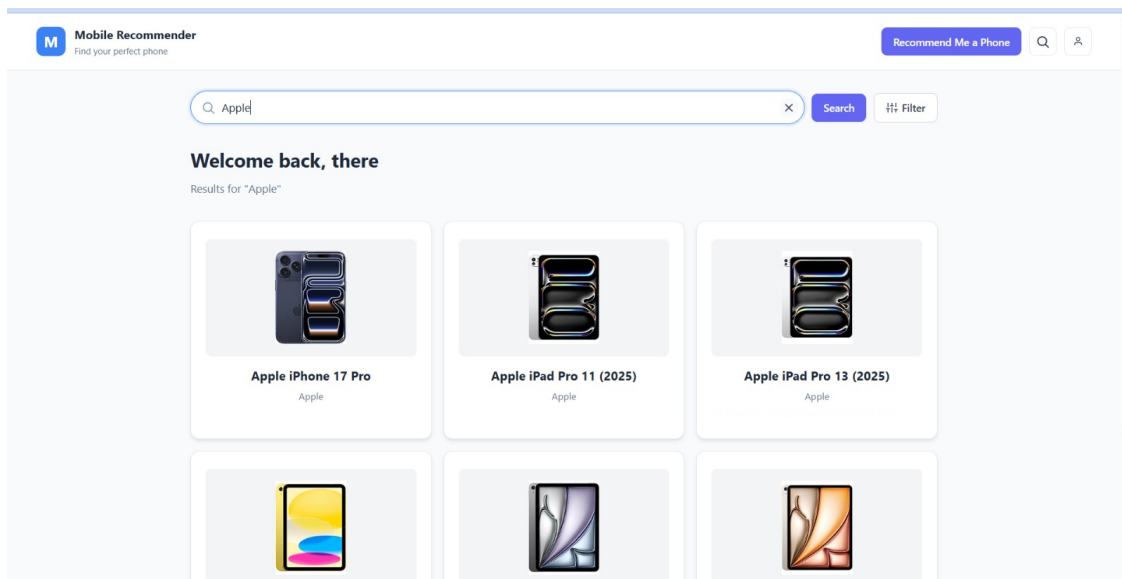


Figure 6.3: Frontend User Interface of the Mobile Recommendation System

The machine learning, frontend, and backend components have been integrated for simple recommendation generation based on user preferences.

6.8 Testing, Evaluation, and Validation

The system was tested in two stages. The first stage validated the trained models on a held-out 20% test split using regression and clustering metrics, as shown in Tables 6.4 and 6.5. The second stage validated the integrated web application through end-to-end tests that mimic the actual user workflow: a customer registers, logs in, sets preferences, and views

recommended phones; a salesman enters customer details and views phone recommendations; and an administrator manages users and views monitoring information.

7. Conclusions

This project successfully demonstrated a complete machine learning based mobile recommendation system that combines customer segmentation with hardware-aware phone scoring. The proposed pipeline integrates web scraping, data preprocessing, feature engineering, XGBoost regression, K-Means clustering, SHAP-based explainability, and a full-stack web application behind a single end-to-end system.

The XGBoost regressor achieved an R^2 score of approximately 0.88 on the held-out test set with a mean absolute error of 0.18 in log-space, indicating that the model predicts the benchmark score of mobile phones with high accuracy. The 5-fold cross-validation produced a mean R^2 of 0.88 with a low standard deviation, which confirms that the model generalises well and is not overfitting to the training data. The K-Means clustering pipeline grouped customers into segments with a strong silhouette score, allowing the recommendation engine to apply segment-specific ranking strategies. The integration of SHAP makes the predictions of the XGBoost model transparent by quantifying the contribution of each phone feature, which is essential for building user trust.

The web application fulfils all functional and non-functional requirements set out in the system design chapter. Customers can register, log in, set preferences, and view recommended phones. Sales staff can enter customer details and receive ranked recommendations. Administrators can manage users and monitor activity. The use of PostgreSQL through Prisma ORM provides a robust and maintainable data layer.

Overall, the project demonstrates that combining a gradient boosting regressor with a clustering-based segmentation pipeline and an explainability layer produces useful, interpretable, and actionable recommendations for mobile phone retailers. The architecture is modular and extensible, which supports future iterations to improve recommendation quality and to incorporate new data sources.

8. Limitations and Future Enhancement

This chapter describes the major limitations of the project and the further enhancements that can be incorporated into the system.

8.1 Limitations

1. The customer transaction dataset is synthetic and does not capture the full noise and distribution of real retailer data.
2. The recommendation engine is a ranker that uses hard filters and a weighted sum of dimension scores; it does not learn from interaction history through collaborative filtering.
3. The benchmark coverage on GSMArena and AnTuTu is incomplete, with missing values for many older or less popular phones.
4. The model is a single XGBoost regressor and does not account for time-based changes in customer preferences.
5. The deployed system does not perform real-time incremental updates; the model is refit offline.

8.2 Future Enhancement

1. Incorporate collaborative filtering and matrix factorisation alongside content-based ranking to address the cold-start problem more comprehensively.
2. Replace the synthetic customer dataset with real retailer data (subject to appropriate consent and privacy controls).
3. Add a temporal component to capture changes in customer preferences and product releases over time.
4. Deploy the system on a containerised platform such as Docker and Kubernetes to simplify scaling.
5. Add A/B testing infrastructure to evaluate the impact of the recommendation system on customer behaviour in production.

References

- [1] ResearchGate. A general architecture of xgboost, 2026. [Online]. Available: https://www.researchgate.net/figure/A-general-architecture-of-XGBoost_fig3_335483097. [Accessed: Jul. 10, 2026].
- [2] Towards AI. Fully explained k-means clustering with python, 2026. [Online]. Available: <https://pub.towardsai.net/fully-explained-k-means-clustering-with-python-e7caa573176a>. [Accessed: Jul. 10, 2026].
- [3] B. Singh. Customer segmentation recommendation. GitHub Repository, 2020. [Online]. Available: <https://github.com/bhaveshsingh0206/customer-segmentation-recommendation>. [Accessed: Feb. 26, 2026].
- [4] GSMArena. Gsmarena – mobile phone specifications, reviews, and news. GSMArena, 2026. [Online]. Available: <https://www.gsmarena.com/>. [Accessed: Feb. 26, 2026].
- [5] AnTuTu. Antutu benchmark – mobile device performance ranking. AnTuTu, 2026. [Online]. Available: <https://www.antutu.com/>. [Accessed: Jul. 10, 2026].
- [6] B. K. Pandey. The cold start problem in recommender systems. freeCodeCamp, 2022. [Online]. Available: <https://www.freecodecamp.org/news/cold-start-problem-in-recommender-systems/>. [Accessed: May 20, 2024].
- [7] T. Chen and C. Guestrin. Xgboost: A scalable tree boosting system. In *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, pages 785–794, 2016.
- [8] Wiley Online Library. Cross-selling precision marketing strategy based on xgboost model and shap explanatory framework. *Wiley Online Library*, 2024.
- [9] N. Işık. A hyper-personalized product recommendation system focused on customer segmentation. *Journal of Theoretical and Applied Electronic Commerce Research*, 18(4), 2023.
- [10] H. Joung and K. Kim. Interpretable machine learning-based approach for customer segmentation. *Technovation*, 127, 2023.

- [11] International Journal of System Assurance Engineering and Management. A shap-enhanced pca-dbscan framework for retail customer segmentation. *International Journal of System Assurance Engineering and Management*, 2025.
- [12] arXiv. Value-aware product recommendation using customer segmentation. *arXiv*, 2024.
- [13] Y. Zhang, B. Pang, Y. Shi, and X. Wang. Large scale purchase prediction with historical user actions. *arXiv*, 2014.
- [14] Predictive Analytics in E-Commerce. Predictive analytics in e-commerce for customer behavior forecasting. 2024.
- [15] Customer Segmentation. Customer segmentation and personalized recommendation system using machine learning. 2024.
- [16] Springer. An explainable ai framework for personalized product recommendation. *Springer*, 2024.
- [17] J. H. Friedman. Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5):1189–1232, 2001.
- [18] S. M. Lundberg and S.-I. Lee. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.
- [19] J. MacQueen. Some methods for classification and analysis of multivariate observations. In *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, volume 1, pages 281–297, 1967.
- [20] P. Resnick, N. Iacovou, M. Suchak, P. Bergstrom, and J. Riedl. Grouplens: An open architecture for collaborative filtering of netnews. In *Proceedings of the 1994 ACM Conference on Computer Supported Cooperative Work*, pages 175–186, 1994.
- [21] M. J. Pazzani and D. Billsus. Content-based recommendation systems. In *The Adaptive Web: Methods and Strategies of Web Personalization*, volume 4321, pages 325–341. Springer, 2007.
- [22] R. Burke. Hybrid recommender systems: Survey and experiments. *User Modeling and User-Adapted Interaction*, 12(4):331–370, 2002.
- [23] B. Sarwar, G. Karypis, J. Konstan, and J. Riedl. Item-based collaborative filtering recommendation algorithms. In *Proceedings of the 10th International Conference on World Wide Web*, pages 285–295, 2001.

- [24] C. D. Manning, P. Raghavan, and H. Schütze. *Introduction to Information Retrieval*. Cambridge University Press, 2008.

Appendices

The following appendices provide supplementary information about the project including the architecture diagrams, workflow descriptions, and project planning notes that supported the implementation.

Appendix A: System Architecture Overview

Content to be added.

Appendix B: ML Workflow Details

Content to be added.

Appendix C: Project Planning Notes

Content to be added.